

Distributed Graph Analytics using Apache Spark GraphX: PageRank, Connected Components, Triangle Counting, and Pregel Reachability Analysis on Barabási–Albert Networks

JIANING LI, University of Waterloo, Canada

This project explores the basic use of Spark GraphX through a series of small experiments on synthetic Barabási–Albert (BA) graphs. The goal was not to design new algorithms, but simply to understand how the main GraphX operations behave in practice, following closely the official documentation and examples. I tested PageRank, Connected Components, Triangle Count, and a small Pregel program for reachability on graphs of different sizes and partition settings. The experiments show how runtime changes with graph size and with the number of partitions, and they also help illustrate the structure of BA graphs. Overall, this project provided a hands-on introduction to distributed graph analytics for someone without a strong computer science background, and it allowed me to evaluate GraphX performance in a simple but systematic way.

Additional Key Words and Phrases: GraphX, Spark, Pregel, PageRank, Triangle Count, Barabási–Albert Graph, Distributed Systems

ACM Reference Format:

Jianing Li. 2025. Distributed Graph Analytics using Apache Spark GraphX: PageRank, Connected Components, Triangle Counting, and Pregel Reachability Analysis on Barabási–Albert Networks. In *Proceedings of Data-Intensive Distributed Computing (CS 651 Final Project)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Graph processing is an important topic in data-intensive systems, as many real-world datasets naturally form graph structures, such as social networks, citation networks, and economic networks. In recent years, distributed graph processing frameworks have become increasingly important because many graphs are too large to be processed on a single machine. Apache Spark GraphX is one such framework that integrates graph computation into the Spark ecosystem.

The goal of this project is not to design new graph algorithms, but to understand how existing GraphX operators behave in practice. I approach this project as a student with an economics background rather than a computer science background. As a result, my focus is on learning how distributed graph analytics works at a high level, how different operators are used, and how performance changes when graph size and partition settings vary.

Instead of working with real-world datasets, I chose to use synthetic Barabási–Albert (BA) graphs. This decision was made mainly for practical reasons. Real datasets often require significant time for data cleaning and preprocessing, which would distract from the main goal of learning GraphX itself. BA graphs are widely used in teaching and research because they are easy to generate and exhibit properties similar to real networks, such as the presence of hub nodes and short path lengths.

Author’s Contact Information: Jianing Li, jianing.li@uwaterloo.ca, University of Waterloo, Waterloo, Ontario, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

This project is organized around two main components. The first component is a reimplementa-
 tion of the *GraphX-Example* from the Spark documentation, using a very small graph. This part helped me understand the basic GraphX
 APIs and verify correctness. The second component is the *FinalProject*, which extends these ideas to larger BA graphs
 and adds timing measurements to compare runtime behavior across different graph sizes and partition counts.

Overall, the project aims to provide a structured learning experience with Spark GraphX, focusing on practical
 experimentation rather than algorithmic novelty.

2 Methodology

The methodology of this project follows a learning-driven and experiment-oriented approach. Instead of starting
 directly with large-scale experiments, I first focused on understanding GraphX at a small scale, and then gradually
 moved toward larger graphs and performance comparison.

2.1 Learning Path and Project Structure

The project consists of two related but distinct programs: `GraphXExample.scala` and `FinalProject.scala`.

The `GraphXExample` program is based on the example provided in the Spark GraphX Programming Guide [1].
 However, it is not a direct copy of the documentation code. Instead, I reimplemented the example step by step while
 reading the documentation, rewriting each operator in my own way to ensure that I understood its behavior. This
 example uses a very small graph defined in `followers.txt`, which makes it easy to manually inspect outputs and
 verify correctness.

Once I felt comfortable with GraphX operators such as `mapVertices`, `subgraph`, `joinVertices`, `PageRank`, `Con-
 nected Components`, and `Triangle Count`, I moved on to the `FinalProject`. This second program uses the same operators
 but applies them to much larger BA graphs and records execution time for comparison.

2.2 Dataset Construction

All experiments in the `FinalProject` use synthetic Barabási–Albert graphs with three different sizes: 10,000, 30,000, and
 50,000 edges. These graphs were generated offline and saved as CSV edge lists. Each dataset was then loaded into Spark
 using GraphX utilities.

BA graphs were chosen because they are sparse and highly connected, making them suitable for demonstrating
 graph algorithms without introducing excessive computational cost.

2.3 Partition Strategy and Implementation Details

During implementation, I learned that GraphX does not allow calling `partitionBy()` directly with an integer, which
 results in a type mismatch error. Instead, an explicit `PartitionStrategy` must be provided. After consulting the Spark
 API documentation, all experiments use:

```
.partitionBy(PartitionStrategy.EdgePartition2D, numPartitions)
```

This follows the recommendations in the Spark documentation regarding graph partitioning [2]. Encountering this
 issue helped reinforce the importance of carefully reading API documentation when working with distributed systems.

2.4 Timing Measurements

Each major GraphX operator was wrapped inside a timing function to measure execution time. The measured components include:

- Graph loading
- PageRank
- Connected Components
- Triangle Count
- Pregel-based reachability

Each experiment was run once due to time constraints, with the goal of identifying general trends rather than precise benchmarks.

3 GraphXExample vs. FinalProject

The GraphXExample and FinalProject programs serve complementary roles in this project.

The GraphXExample uses a very small graph defined in `followers.txt`. Because the graph is small, its primary purpose is correctness and understanding rather than performance. This allowed me to inspect intermediate outputs and verify that each GraphX operator behaved as expected. For example, I could manually check connected components and triangle counts.

In contrast, the FinalProject applies the same operators to much larger BA graphs. At this scale, it becomes impractical to manually inspect results, so the focus shifts to runtime behavior and performance comparison. The FinalProject also introduces command-line parameters (using Scallop) to control the number of partitions and the source node for Pregel reachability.

This comparison highlights an important lesson: small example graphs are useful for learning APIs, while larger synthetic graphs are better suited for performance observation.

4 Results

The experimental results show consistent and interpretable patterns. PageRank was the most expensive operation, followed by Triangle Count, while Connected Components and Pregel reachability were relatively fast. Increasing graph size led to higher runtime, but the increase was not proportional to graph size due to the sparsity of BA graphs.

Changing the number of partitions did not always improve performance. In several cases, increasing partitions from 16 to 32 slightly increased runtime, likely due to additional communication overhead. This observation matches the discussion in the Spark documentation regarding partitioning tradeoffs [1].

Across all experiments, almost all vertices were reachable from the chosen source node. This confirms the expected structure of BA graphs, which typically form one giant connected component.

4.1 Execution Time Comparison

Table 1 shows execution times for all operators under different graph sizes, using 8 partitions.

To isolate the effect of partitioning, Table 2 reports average runtime across all graph sizes for different partition counts.

4.2 Effect of Graph Size

Table 1 shows that increasing graph size leads to higher runtime for all GraphX operators. However, the increase is moderate rather than linear. For example, PageRank increases from about 4 seconds on the 10k graph to around 5.6 seconds on the 50k graph. This suggests that GraphX handles sparse BA graphs efficiently, and that graph size alone is not the dominant performance bottleneck at this scale.

Connected Components shows only a small increase as graph size grows, remaining close to one second even for the largest graph. Triangle Count grows more noticeably with graph size, but still remains under 2.5 seconds for the 50k graph. Pregel reachability scales smoothly as well, increasing from roughly 0.75 seconds to just under 1 second.

Overall, these results suggest that GraphX algorithms scale reasonably well with graph size for sparse networks such as BA graphs.

4.3 Effect of Partition Count

Table 2 isolates the effect of changing the number of partitions. The results show that increasing partitions does not always improve performance. For several operators, moving from 8 to 16 partitions leads to only minor changes, while increasing to 32 partitions sometimes increases runtime.

This behavior can be explained by overhead communication and coordination overhead. For relatively small graphs, the cost of managing more partitions may outweigh the benefits of additional parallelism. This effect is most visible for PageRank and Pregel reachability, where 32 partitions consistently result in slower execution.

These observations align with the Spark documentation, which notes that partitioning must be chosen carefully and that more partitions are not always better, especially for smaller or sparse datasets.

5 Pregel Reachability and Dead-Node Detection

The Pregel API was the most conceptually challenging part of this project. After implementing reachability, I realized that running the algorithm once per node could detect dead-end nodes. This led to a second Pregel design (Pregel 2).

However, since Pregel 1 takes around 0.8–1.2 seconds per run, running it for all nodes would require several hours for large graphs. As a result, Pregel 2 was tested only on the small example graph, and its cost on BA graphs is discussed conceptually rather than experimentally.

5.1 Estimated Cost of Pregel Dead-Node Detection

Pregel 2 attempts to detect dead-end nodes by running the reachability algorithm once for each vertex. Although this approach is conceptually simple, it is computationally expensive.

Based on the measured runtime of Pregel 1, which takes approximately 0.8–1.2 seconds per run depending on graph size, the total cost of Pregel 2 can be estimated by multiplying this runtime by the number of vertices. Table 3 summarizes the estimated cost.

Given these estimates, running Pregel 2 on BA graphs would exceed the available time budget for this project. Therefore, the algorithm was only tested on the small example graph used in GraphXExample, and the design is included here for completeness rather than full experimental evaluation.

6 Discussion

This project provided several practical insights into how GraphX behaves when applied to synthetic Barabási–Albert graphs. An important observation is that increasing the number of partitions does not always improve performance. Although more partitions increase parallelism in theory, the experiments showed that for relatively small and sparse graphs, the overhead of communication and coordination can outweigh the benefits. This was especially visible when moving from 16 to 32 partitions, where several operators became slightly slower instead of faster.

Another key lesson from this project is the importance of starting with small, understandable examples. Rebuilding the `GraphXExample` before running large experiments was crucial to my learning process. Working with a tiny graph made it possible to manually inspect outputs and understand how each operator works. Without this step, it would have been much harder to reason about the behavior of the algorithms on larger graphs, where the output is no longer easy to interpret.

The Pregel experiments were particularly educational. The reachability algorithm (Pregel 1) demonstrated how quickly information can propagate through BA graphs, reflecting their highly connected structure. At the same time, the attempt to design a dead-end node detection algorithm (Pregel 2) highlighted an important limitation of naive approaches in distributed systems. Running a full Pregel computation once per node is conceptually simple, but it quickly becomes impractical as the graph size increases.

If more time and computational resources were available, there are several ways this part of the project could be improved. For example, instead of running Pregel separately for each node, it may be possible to redesign the algorithm to detect dead-end nodes using a single or a small number of Pregel runs, like what we did in the PageRank course assignment. Exploring these ideas would require a deeper understanding of graph algorithms and more experimentation, and they could be interesting directions for future work.

Overall, the discussion highlights a recurring theme of this project: simple ideas are often easy to implement, but making them scalable in a distributed setting requires careful design and trade-offs.

7 Conclusion

This project explored distributed graph analytics using Apache Spark GraphX from a practical, learning-oriented perspective. Rather than focusing on designing new algorithms, the project aimed to understand how existing GraphX operators behave when applied to graphs of different sizes and under different partition settings.

By combining a small-scale re-implementation of the `GraphXExample` with larger experiments on synthetic Barabási–Albert graphs, I was able to build intuition about both correctness and performance. The experiments showed clear and consistent patterns: PageRank is the most expensive operation, Connected Components is very fast, Triangle Count scales moderately with graph size, and Pregel-based reachability performs efficiently on highly connected graphs. The results also demonstrated that increasing the number of partitions does not automatically improve performance, especially for sparse graphs.

From a learning standpoint, this project helped bridge the gap between documentation and practice. Concepts described in the Spark GraphX documentation became much clearer after implementing and timing them in real experiments. The Pregel API, in particular, illustrated both the power and the limitations of message-passing models in distributed systems.

Although the project has limitations—such as running each experiment only once and not fully evaluating the dead-node detection algorithm—it still achieved its main goal. It provided hands-on experience with distributed graph

processing and highlighted important trade-offs that arise in practice. Overall, this project serves as a solid foundation for further exploration of large-scale graph analytics and distributed systems.

Acknowledgments

I thank the course staff for guidance and the Spark GraphX documentation for reference examples.

References

- [1] Apache Spark. 2024. *GraphX Programming Guide*. Apache Software Foundation. <https://spark.apache.org/docs/latest/graphx-programming-guide.html> Accessed: 2025-03-.
- [2] Apache Spark. 2024. *PartitionStrategy (GraphX Scala API)*. Apache Software Foundation. [https://spark.apache.org/docs/latest/api/scala/org/apache/spark/graphx/PartitionStrategy\\$.html](https://spark.apache.org/docs/latest/api/scala/org/apache/spark/graphx/PartitionStrategy$.html) Accessed: 2025-03-.

Table 1. Runtime (seconds) by graph size using 8 partitions.

Graph Size	PageRank	CC	Triangle Count	Pregel
10k	4.10	1.00	1.05	0.75
30k	5.05	1.24	1.64	0.75
50k	5.60	1.50	2.25	0.95

Table 2. Average runtime (seconds) across all graph sizes under different partition counts.

Partitions	PageRank	CC	Triangle Count	Pregel
8	4.92	1.25	1.65	0.82
16	4.90	1.27	1.64	0.93
32	5.46	1.45	1.56	1.18

Table 3. Estimated runtime of Pregel dead-node detection (Pregel 2).

Graph Size	Pregel 1 (sec)	Estimated Pregel 2
10k	~0.75	~2 hours
30k	~0.85	~7 hours
50k	~1.00	~14 hours

8 Figures

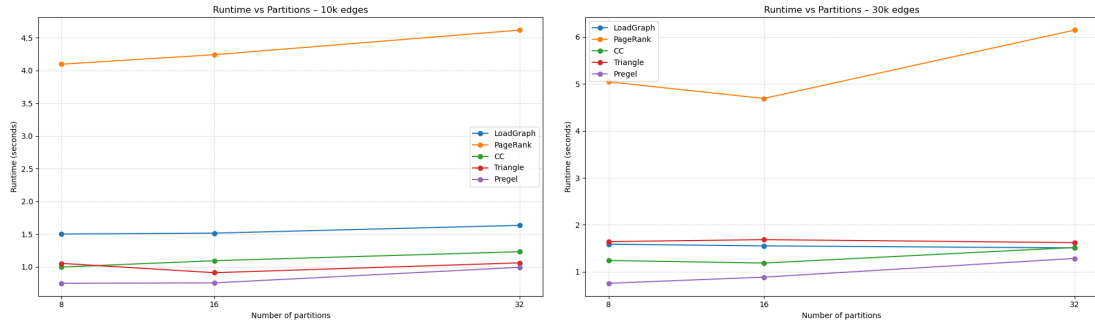


Fig. 1. Runtime comparison for 10k and 30k BA graphs under different operators.

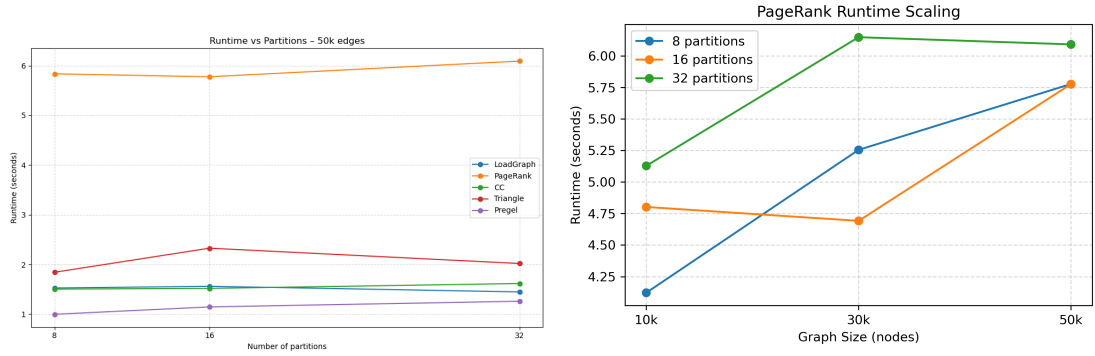


Fig. 2. Runtime comparison for the 50k BA graph and PageRank partition scaling.

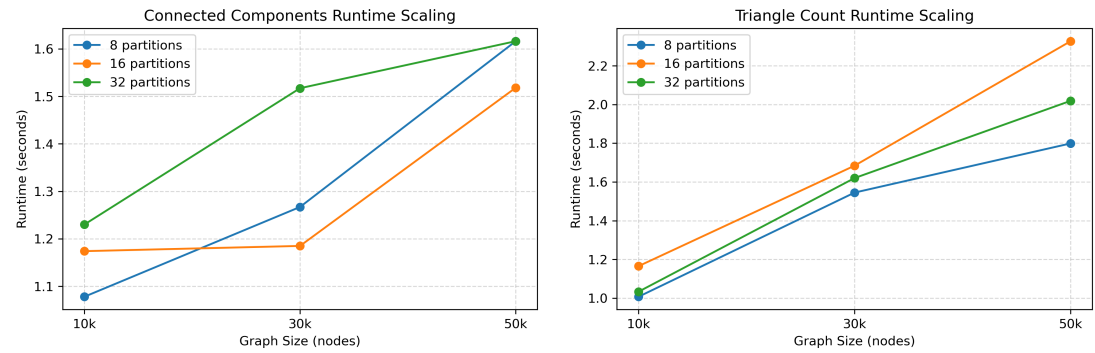


Fig. 3. Connected Components and Triangle Count runtime under different partition settings.

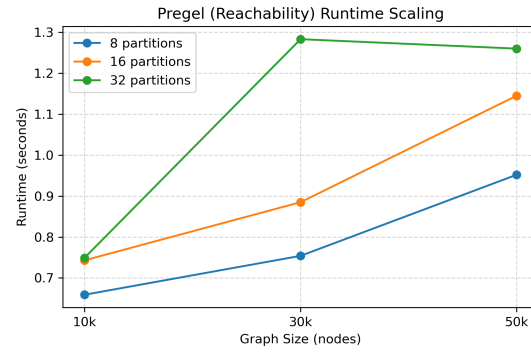


Fig. 4. Pregel reachability runtime under different partition settings.